



FVGridMaker

structured grids for finite volumes

Manual da Biblioteca

Geração de malhas estruturadas para Volumes Finitos

Biblioteca em C++20 para a geração de malhas estruturadas (1D e 2D)
para uso em programas baseados no Método dos Volumes Finitos.

Sumário

Apresentação	3
Malha 1D	5
Malha Uniforme	6
Malha de Roberts	10
Malha Aleatória	12
Malha Customizada	14
Malha Lei de Potência	16
Centros Ponderados	18
Métricas de Qualidade	20
Exportação CSV	23
Malha 2D	25
Malha 2D Cartesiana	26
Numeração em malha 2D	30
Faces explícitas	32
Campo escalar 2D	34
Malha Polar 2D	37
Inspeção da malha 2D	40
Malha 2D: Qualidade e VTK	42
Coordenadas Axissimétricas	44
Anular com Setor Removido	46
Malha Elíptica Confocal	48
Malha Elíptica Confocal	50

Apresentação

A FVGridMaker é uma biblioteca C++ para a geração de malhas estruturadas em uma e duas dimensões, voltada a programas baseados no Método dos Volumes Finitos. Seu objetivo não é tratar geometrias complexas nem substituir geradores gerais de malha. O escopo é mais restrito: construir malhas simples, regulares ou não uniformes, nas quais faces, centros, volumes, áreas, índices e vizinhanças possam ser consultados diretamente pelo código numérico.

A biblioteca foi escrita com recursos de C++ moderno para reduzir a repetição de detalhes geométricos de baixo nível. Em vez de o usuário recalcular coordenadas, índices ou medidas geométricas em cada programa, a FVGridMaker concentra essas informações numa interface própria para uso em discretizações por volumes finitos.

Este manual apresenta exemplos curtos e reproduzíveis. O primeiro exemplo é mais detalhado, pois fixa o padrão de uso da biblioteca. Nos exemplos seguintes, o texto passa a destacar apenas o que muda: uma nova forma de geração da malha, uma nova consulta geométrica ou uma nova organização dos dados.

Caso encontre erro, comportamento inesperado ou dificuldade técnica, envie uma mensagem para livromvf@gmail.com, preferencialmente com uma descrição do problema, o exemplo utilizado e a versão da biblioteca.

Como executar os exemplos do manual

Os programas usados neste manual estão no diretório `manual/`. Eles seguem a convenção de nomes `man_*.cc` e devem ser executados a partir da raiz do repositório, isto é, do diretório onde está o arquivo `CMakeLists.txt` principal.

A forma recomendada de compilação e execução é pelo CMake. Primeiro, configure o projeto:

configuração

```
cmake -S . -B build
```

Depois, compile a biblioteca e os exemplos:

compilação

```
cmake --build build
```

Cada arquivo do manual gera um alvo de execução. Por exemplo,

convenção

```
manual/man_uniformGrid.cc -> run_man_uniformGrid
```

Logo, para executar esse exemplo, use:

execução

```
cmake --build build --target run_man_uniformGrid
```

Para executar todos os exemplos do manual de uma vez, use:

todos os exemplos

```
cmake --build build --target run_all_manual_programs
```

atenção

Windows

No Windows nativo, recomenda-se usar o Visual Studio 2022 ou o Build Tools for Visual Studio. Nesse caso, a configuração pode ser feita com `cmake -S . -B build -G "Visual Studio 17 2022" -A x64`. Ao compilar ou executar um alvo, informe também a configuração, por exemplo: `cmake --build build --config Debug --target run_man_uniformGrid`.

atenção

Visual Studio Code

No Visual Studio Code, abra a raiz do repositório e use a extensão CMake Tools. Escolha um kit de compilação, configure o projeto e execute os alvos `run_man_*`. Evite executar diretamente os arquivos `.cc` por botões genéricos de execução, pois esse caminho costuma ignorar a configuração real do projeto.

observação

Diretório de execução

Não é necessário entrar manualmente no diretório `build`. A opção `-B build` informa ao CMake onde ficam os arquivos de compilação, e `cmake --build build` usa esse diretório diretamente. Quando algum exemplo grava um arquivo de saída, o próprio programa deve informar no terminal o caminho gerado.

01

Malha 1D

faces, centros, volumes e inspeção mínima

Os exemplos deste capítulo apresentam os recursos fundamentais da FVGridMaker para malhas unidimensionais. A sequência começa pela malha uniforme, tratada com mais detalhe por estabelecer o padrão de uso da biblioteca, e avança para variações que introduzem apenas uma novidade por vez: distribuição não uniforme, coordenadas fornecidas pelo usuário, reconstrução de centros, métricas de qualidade e exportação de dados.

Recomenda-se executar os programas na ordem em que aparecem. A interface 1D fixa conceitos que serão reutilizados nas malhas bidimensionais, como faces, centros, comprimentos, inspeção geométrica e organização dos dados.

seção 1.1

Construindo uma malha uniforme

O primeiro exemplo constrói uma malha cartesiana uniforme em uma dimensão. O domínio é o intervalo $([0,1])$, dividido em (10) volumes finitos de mesmo comprimento. O objetivo é gerar a malha, imprimir suas informações geométricas básicas e verificar se a soma dos comprimentos dos volumes reproduz o comprimento total do domínio.

A malha uniforme é o ponto de partida natural porque sua geometria pode ser conferida diretamente. Se o domínio tem comprimento (1) e foi dividido em (10) volumes, cada volume deve ter comprimento (0,1). Essa previsibilidade permite verificar se a biblioteca foi chamada corretamente antes de avançar para malhas não uniformes.

Estrutura básica do programa

Vamos destacar as partes principais do arquivo `man_uniformGrid.cc`. O programa completo pode ser acessado pelo QR code ao final da seção. Para usar a FVGridMaker, o exemplo inclui o cabeçalho público principal da biblioteca:

cabeçalho da FVGridMaker

```
1 #include <FVGridMaker/FVGridMaker.h>
```

No arquivo completo também aparecem cabeçalhos da biblioteca padrão de C++ , como `<iostream>`, `<iomanip>`, `<cmath>` e `<string>`. Eles são usados apenas para impressão, formatação e verificação numérica.

O exemplo adota o tipo real padrão da biblioteca e define um apelido para o eixo unidimensional:

tipo escalar e eixo 1D

```
1 using Scalar = fvgrid::Real;  
2 using Axis = fvgrid::BasicAxis1D<Scalar>;
```

O tipo `fvgrid::Real` representa o tipo real padrão da FVGridMaker. Já o seguinte comando `fvgrid::BasicAxis1D<Scalar>` representa um eixo unidimensional parametrizado pelo tipo numérico. Se for necessário usar outra precisão, basta trocar o tipo escalar, por exemplo:

alterando a precisão

```
1 using Scalar = long double;  
2 using Axis = fvgrid::BasicAxis1D<Scalar>;
```

O restante do programa permanece essencialmente igual.

Definição e geração da malha

Uma malha uniforme unidimensional é definida pelo número de volumes e pelos limites do domínio:

dados da malha

```
1 const fvgrid::Size nvol = 10;  
2 const Scalar xinit = Scalar{0.0};  
3 const Scalar xfinal = Scalar{1.0};
```

Aqui, o domínio é $([0,1])$ e contém (10) volumes. Como a malha é uniforme, esses três dados determinam todas as faces, todos os centros e todos os comprimentos. A construção da malha é feita por:

geração da malha

```
1 const Axis axis = fvgrid::uniform_axis_1d<Scalar>(  
2   nvol,  
3   xinit,  
4   xfinal  
5 );
```

Depois dessa chamada, o objeto `axis` passa a armazenar a geometria do eixo. O usuário não precisa reconstruir manualmente as faces, os centros ou os comprimentos dos volumes.

A primeira inspeção pode ser feita imprimindo diretamente o objeto:

impressão automática

```
1 std::cout << std::fixed << std::setprecision(6);  
2 std::cout << axis << '\n';
```

A primeira linha fixa a impressão com seis casas decimais. A segunda usa a representação textual padrão do eixo, útil para uma conferência rápida durante o desenvolvimento.

Consulta das propriedades geométricas

O objeto `axis` fornece funções para consultar as propriedades globais da malha:

resumo da malha

```
1 std::cout << "número de volumes : " << axis.num_cells() << '\n';  
2 std::cout << "número de faces   : " << axis.num_faces() << '\n';  
3 std::cout << "xmin              : " << axis.xmin() << '\n';  
4 std::cout << "xmax              : " << axis.xmax() << '\n';  
5 std::cout << "comprimento       : " << axis.length() << '\n';
```

A função `axis.num_cells()` retorna o número de volumes, enquanto `axis.num_faces()` retorna o número de faces. Em uma malha 1D com (N) volumes, há (N+1) faces. As funções `axis.xmin()`, `axis.xmax()` e `axis.length()` retornam, respectivamente, o limite esquerdo, o limite direito e o comprimento total do domínio.

As coordenadas das faces são acessadas por índice:

coordenadas das faces

```
1 for (fvgrid::Size i = 0; i < axis.num_faces(); ++i) {
2   std::cout << std::right
3   << std::setw(id_width) << i
4   << std::setw(value_width) << axis.face(i)
5   << '\n';
6 }
```

Como o exemplo usa (10) volumes, a malha possui (11) faces. A chamada `axis.face(i)` retorna a coordenada da face de índice (i).

As informações associadas a cada volume são consultadas pelo índice (p):

informações dos volumes

```
1 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
2   std::cout << std::right
3   << std::setw(id_width) << p
4   << std::setw(value_width) << axis.west_face(p)
5   << std::setw(value_width) << axis.center(p)
6   << std::setw(value_width) << axis.east_face(p)
7   << std::setw(value_width) << axis.cell_length(p)
8   << '\n';
9 }
```

A chamada `axis.west_face(p)` retorna a face oeste do volume (p), `axis.east_face(p)` retorna a face leste, `axis.center(p)` retorna o centro e `axis.cell_length(p)` retorna o comprimento do volume. Para uma malha uniforme no intervalo $[0,1]$ com (10) volumes, todos os valores de Δx_p devem ser iguais a 0,1.

Teste de consistência

O exemplo termina com uma verificação geométrica simples. A soma dos comprimentos dos volumes deve reproduzir o comprimento total do domínio:

$$\sum_{p=0}^{N-1} \Delta x_p = x_{\max} - x_{\min}.$$

No código, a soma é calculada percorrendo todos os volumes:

teste de consistência

```
1 Scalar sum_dx = Scalar{0.0};
2
3 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
4   sum_dx += axis.cell_length(p);
5 }
6
7 const Scalar expected_length = axis.length();
8 const Scalar error = std::abs(sum_dx - expected_length);
9 const Scalar tolerance = Scalar{1.0e-12};
```


Como os cálculos usam ponto flutuante, o programa compara o erro com uma tolerância pequena, em vez de exigir igualdade exata. Esse teste é simples, mas fixa um padrão que será reutilizado nos exemplos seguintes, especialmente nas malhas não uniformes, onde os comprimentos deixam de ser todos iguais.

observação**Padrão para os próximos exemplos**

Este primeiro exemplo apresenta a nomenclatura básica usada no restante do manual. As faces delimitam os volumes, os centros indicam as posições associadas aos volumes, e o comprimento Δx_p mede a extensão de cada volume no eixo x . Nos exemplos seguintes, essas ideias não serão repetidas em detalhe; o texto destacará apenas as diferenças em relação a esta malha uniforme.

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker. Aponte a câmera do celular para o QR code abaixo para acessar o arquivo.



seção 1.2

Construindo uma malha de Roberts

A malha de Roberts é o primeiro exemplo não uniforme do manual. Originalmente proposta por Roberts [2] e apresentada no contexto de volumes finitos por Maliska [1], ela permite concentrar volumes de forma suave em certas regiões do domínio, sem introduzir saltos aleatórios entre volumes vizinhos.

A diferença em relação à malha uniforme está na distribuição das faces. No exemplo anterior, as faces eram igualmente espaçadas. Aqui, elas são obtidas por uma transformação algébrica controlada por um parâmetro β . Depois que o eixo é construído, o acesso à geometria permanece o mesmo: faces, centros e comprimentos continuam sendo consultados pelo objeto `axis`.

A equação usada é

$$\xi(\eta) = \frac{x(\eta)}{L} = \frac{(\beta + 1)A(\eta) - (\beta - 1)}{2[A(\eta) + 1]}, \quad \text{com} \quad A(\eta) = \left(\frac{\beta + 1}{\beta - 1}\right)^{2\eta - 1} \quad (1)$$

e

$$0 \leq \eta \leq 1, \quad \beta > 1.$$

Quanto mais β se aproxima de 1, maior é a concentração junto às fronteiras. Para valores maiores de β , a malha se aproxima de uma distribuição mais uniforme.

Novidade do exemplo

O arquivo correspondente é `man_robertsGrid.cc`. A estrutura geral já é conhecida: define-se o domínio, constrói-se o eixo, imprime-se a geometria e verifica-se o fechamento dos comprimentos. A novidade está na chamada de construção da malha:

geração da malha de Roberts

```
1  const Axis axis = fvgrid::roberts_axis_1d(
2  fvgrid::NVol{nv},
3  fvgrid::Length{xfinal - xinit},
4  fvgrid::XInit{xinit},
5  fvgrid::Beta{beta},
6  fvgrid::VolumeCentered1D{}
7  );
```

O parâmetro `fvgrid::Beta{beta}` controla a intensidade da concentração. A opção empregada `fvgrid::VolumeCentered1D{}` mantém o mesmo padrão conceitual usado no exemplo anterior: as faces são as coordenadas primárias, e os centros dos volumes são reconstruídos a partir delas.

O que observar

Depois da construção do eixo, nada muda na forma de consultar a geometria. Chamadas como `axis.west_face(p)`, `axis.center(p)`, `axis.east_face(p)` e `axis.cell_length(p)` continuam

tendo o mesmo significado que tinham na malha uniforme.

O que muda são os valores impressos. Na malha uniforme, todos os comprimentos Δx_p eram iguais. Na malha de Roberts, os comprimentos variam suavemente ao longo do domínio. Por isso, este exemplo funciona como uma ponte entre a malha uniforme e a malha aleatória: ele introduz não uniformidade, mas ainda com variação controlada.

O programa também calcula indicadores simples de concentração:

indicadores de concentração

```
1 const Scalar concentration_ratio = max_dx / min_dx;
2
3 std::cout << "menor dx          : " << min_dx << '\n';
4 std::cout << "maior dx         : " << max_dx << '\n';
5 std::cout << "razão maior/menor dx : " << concentration_ratio << '\n';
```

Esses indicadores não dizem se a malha é adequada para qualquer problema físico ou numérico. Eles apenas quantificam quanto a malha se afastou do caso uniforme.

observação

Malha não uniforme suave

A malha de Roberts deve ser lida como a primeira variação da malha uniforme. A interface de consulta não muda; apenas a distribuição das faces deixa de ser uniforme. Essa separação entre construção da malha e consulta da geometria é uma das ideias centrais da FVGridMaker.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_robertsGrid
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 1.3

Construindo uma malha aleatória

O exemplo `man_randomGrid.cc` fecha o primeiro grupo de malhas 1D: uniforme, Roberts e aleatória. O domínio continua sendo $[0,1]$, dividido em (10) volumes, mas agora as faces internas são sorteadas no interior do intervalo. As faces externas permanecem fixas em $(x=0)$ e $(x=1)$.

A diferença em relação aos exemplos anteriores está na regularidade da distribuição. A malha uniforme tem comprimentos iguais; a malha de Roberts tem variação suave e controlada; a malha aleatória quebra essa regularidade. Por isso, ela é útil como teste de estresse: se um programa funciona apenas quando Δx é constante ou suavemente variável, esse exemplo tende a expor a hipótese escondida.

Novidade do exemplo

A novidade é a presença de uma semente de geração aleatória:

domínio e semente

```
1 const fvgrid::Size nvol = 10;  
2 const Scalar xinit = Scalar{0.0};  
3 const Scalar xfinal = Scalar{1.0};  
4 const fvgrid::UInt64 seed = 20260627;
```

A semente fixa faz o programa gerar a mesma malha em execuções diferentes. Isso é importante no manual, pois torna o exemplo reproduzível. Se a semente for alterada, as faces internas sorteadas também podem mudar.

Trecho essencial

A construção da malha é feita por `random_axis_1d`:

geração da malha aleatória

```
1 const Axis axis = fvgrid::random_axis_1d<Scalar>(  
2   nvol,  
3   xinit,  
4   xfinal,  
5   seed  
6 );
```

A função mantém as extremidades do domínio fixas, sorteia as faces internas e organiza essas coordenadas em ordem crescente. Depois disso, a FVGridMaker calcula os centros e os comprimentos dos volumes. A partir desse ponto, a interface de consulta é a mesma dos exemplos anteriores: `axis.face(i)`, `axis.center(p)` e `axis.cell_length(p)` continuam tendo o mesmo significado.

O que observar

Na saída, os comprimentos Δx_p variam de forma irregular. Esse comportamento distingue a malha aleatória da malha de Roberts, na qual a variação é suave. O fechamento geométrico continua obrigatório: a soma dos comprimentos dos volumes deve reproduzir $x_{\max} - x_{\min}$.

O programa também imprime a semente usada:

registro da semente

```
1 std::cout << "semente" << " : " << seed << '\n';
```

Esse valor faz parte do caso de teste. Ao registrar a semente, torna-se possível reconstruir a mesma malha posteriormente.

observação

Malha aleatória como teste

A malha aleatória não deve ser tomada automaticamente como uma boa escolha numérica. Neste manual, sua função principal é verificar se o código realmente usa a geometria fornecida pela malha, em vez de assumir implicitamente que os volumes têm o mesmo comprimento.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_randomGrid
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 1.4

Malha com coordenadas próprias

O exemplo `man_customCoordinates1D.cc` mostra como construir uma malha a partir de coordenadas fornecidas pelo usuário. Isso é útil quando as posições vêm de um arquivo, de uma tabela, de uma rotina externa ou de uma regra própria não embutida na biblioteca.

A diferença em relação aos exemplos anteriores está na origem das coordenadas. Antes, a FVGridMaker gerava as faces por uma regra pronta; aqui, as coordenadas já chegam definidas. A biblioteca apenas organiza esses dados em um objeto `Axis1D`, mantendo a mesma interface de consulta para faces, centros e comprimentos.

Novidade do exemplo

O exemplo apresenta dois usos: fornecer faces ou fornecer centros. Quando as faces são conhecidas, elas são passadas como coordenadas primárias:

malha customizada por faces

```
1  std::vector<Scalar> user_faces{
2  0.00, 0.04, 0.10, 0.19, 0.31, 0.46, 0.64, 0.79, 0.90, 0.97, 1.00
3  };
4
5  const Axis axis_from_faces = fvgrid::custom_axis_1d(
6  fvgrid::BasicCoordinates1D<Scalar>::faces(std::move(user_faces)),
7  fvgrid::VolumeCentered1D{}
8  );
```

Como foram fornecidas (11) faces, a malha possui (10) volumes. As coordenadas devem estar em ordem crescente e devem definir volumes com comprimento positivo. O padrão `VolumeCentered1D` indica que os centros serão calculados a partir das faces.

Fornecendo centros

Também é possível construir a malha a partir dos centros dos volumes. Nesse caso, os limites do domínio precisam ser informados separadamente, pois a biblioteca precisa conhecer as faces externas:

malha customizada por centros

```
1 std::vector<Scalar> user_centers{
2   0.03, 0.09, 0.17, 0.28, 0.42, 0.58, 0.72, 0.83, 0.91, 0.97
3 };
4
5 const auto domain = fvgrid::BasicDomain1D<Scalar>::from_bounds(
6   fvgrid::BasicXInit<Scalar>{xinit},
7   fvgrid::BasicXFinal<Scalar>{xfinal}
8 );
9
10 const Axis axis_from_centers = fvgrid::custom_axis_1d(
11   fvgrid::BasicCoordinates1D<Scalar>::centers(std::move(user_centers)),
12   fvgrid::FaceCentered1D{},
13   domain
14 );
```

Aqui, os centros são as coordenadas primárias, e as faces são reconstruídas. Depois da construção, o objeto `Axis1D` é usado como nos exemplos anteriores, com chamadas como `axis.face(i)`, `axis.center(p)` e `axis.cell_length(p)`.

observação

Coordenadas primárias

A palavra “primária” indica apenas qual conjunto de coordenadas foi fornecido primeiro. Ela não significa que faces ou centros sejam mais importantes fisicamente.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_customCoordinates1D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 1.5

Lei de potência

O exemplo `man_powerLawGrid.cc` mostra como gerar uma malha a partir de uma função de mapeamento. A coordenada computacional $\eta \in [0, 1]$ é transformada em uma coordenada física x , e as faces obtidas são entregues à FVGridMaker para a construção do eixo.

A novidade em relação à seção anterior é que as coordenadas não são escritas manualmente. Elas são produzidas por uma regra matemática. Neste exemplo, usa-se a lei de potência

$$x(\eta) = x_{\min} + (x_{\max} - x_{\min}) \eta^{\gamma}.$$

Para $\gamma > 1$, as faces ficam mais concentradas perto de $x_{\min}$. Com $\gamma = 2$, os primeiros volumes são menores e os últimos são maiores.

Novidade do exemplo

O programa define o domínio, o número de volumes e o expoente da transformação:

parâmetros e mapeamento

```
1 const fvgrid::Size nvol = 10;
2 const Scalar xinit = Scalar{0.0};
3 const Scalar xfinal = Scalar{1.0};
4 const Scalar gamma = Scalar{2.0};
5 const Scalar length = xfinal - xinit;
6
7 const auto power_law_map = [xinit, length, gamma](Scalar eta) {
8     return xinit + length * std::pow(eta, gamma);
9 };
```

A função `power_law_map` recebe η e devolve a posição física correspondente. Assim, a distribuição das faces fica concentrada numa função, em vez de aparecer como uma lista explícita de coordenadas.

Trecho essencial

As faces são geradas pelo padrão `VolumeCentered1D` e depois usadas para montar o eixo:

faces por mapeamento

```
1 std::vector<Scalar> faces =
2     fvgrid::VolumeCentered1D::primary_coordinates_from_map<Scalar>(
3         nvol,
4         power_law_map
5     );
6
7 const Axis axis = fvgrid::custom_axis_1d(
8     fvgrid::BasicCoordinates1D<Scalar>::faces(std::move(faces)),
9     fvgrid::VolumeCentered1D{}
10 );
```


A primeira chamada gera $N + 1$ faces para N volumes. A segunda constrói o objeto `Axis1D` a partir dessas faces. Depois disso, a consulta da geometria é igual à dos exemplos anteriores.

O que observar

Na saída, os valores de Δx_p devem aumentar ao longo do domínio. Isso confirma que a função $x = \eta^\gamma$, com $\gamma = 2$, concentrou mais faces perto da origem.

observação

Mapeamento

A função de mapeamento não fica armazenada no eixo. Ela é usada apenas para gerar as faces. Depois da construção, a malha é representada pelas faces, centros e comprimentos calculados.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_powerLawGrid
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 1.6

Centros ponderados

O exemplo `man_alternatingWeightedCenters1D.cc` muda o foco dos exemplos anteriores. Até aqui, variamos principalmente a distribuição das faces. Agora, as faces continuam vindo de uma malha uniforme, mas os centros dos volumes são reconstruídos por uma regra de pesos alternados. Este exemplo não é uma recomendação geral de malha para problemas físicos. Ele serve para mostrar uma separação importante da FVGridMaker: uma coisa é definir as coordenadas primárias da malha; outra é escolher a regra usada para completar a geometria.

Novidade do exemplo

O programa começa com faces uniformes, copiadas de uma malha auxiliar:

faces de partida

```
1  const Axis uniform_axis =
2      fvgrid::uniform_axis_1d<Scalar>(nvol, xinit, xfinal);
3
4  std::vector<Scalar> faces;
5  faces.reserve(uniform_axis.num_faces());
6
7  for (fvgrid::Size i = 0; i < uniform_axis.num_faces(); ++i) {
8      faces.push_back(uniform_axis.face(i));
9  }
```

A malha final terá os mesmos comprimentos da malha uniforme. O que muda é a posição dos centros dentro de cada volume.

Trecho essencial

A regra de reconstrução é definida por uma função que alterna o peso usado em volumes pares e ímpares:

pesos alternados

```
1  const Scalar alpha = Scalar{0.20};
2
3  const auto alternating_weight = [alpha](fvgrid::Size p) {
4      if (p % 2 == 0) {
5          return alpha;
6      }
7      return Scalar{1.0} - alpha;
8  };
9
10 const Axis axis = fvgrid::custom_axis_1d(
11     fvgrid::BasicCoordinates1D<Scalar>::faces(std::move(faces)),
12     fvgrid::CentersFromFaces1D{alternating_weight}
13 );
```

As faces são fornecidas como coordenadas primárias. O padrão `CentersFromFaces1D` usa a função `alternating_weight` para posicionar os centros. Nos volumes pares, o centro fica na fração α do volume. Nos volumes ímpares, fica na fração $1 - \alpha$.

O que observar

Como as faces vêm de uma malha uniforme, os valores de Δx_p continuam iguais. A diferença aparece nos centros: eles deixam de coincidir com os pontos médios dos volumes e passam a alternar de posição.

observação

Faces e centros

O comprimento do volume depende das faces. A posição do centro depende da regra de reconstrução. Neste exemplo, os comprimentos permanecem uniformes, mas os centros deixam de ficar nos pontos médios.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_alternatingWeightedCenters1D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 1.7

Métricas de qualidade

Depois de construir várias malhas, convém inspecionar algumas medidas simples de qualidade. O exemplo `man_quality1D.cc` compara três casos já conhecidos: malha uniforme, malha de Roberts e malha aleatória.

Essas métricas não decidem, sozinhas, se uma malha é adequada para um problema numérico. Elas servem como alerta inicial, útil para detectar erros grosseiros, concentrações excessivas ou saltos fortes entre volumes vizinhos.

Novidade do exemplo

O programa cria três eixos com o mesmo domínio e o mesmo número de volumes:

malhas comparadas

```
1  const Axis uniform_axis =  
2  fvgrid::uniform_axis_1d<Scalar>(nvol, xinit, xfinal);  
3  
4  const Axis roberts_axis = fvgrid::roberts_axis_1d(  
5  fvgrid::NVol{nvol},  
6  fvgrid::Length{xfinal - xinit},  
7  fvgrid::XInit{xinit},  
8  fvgrid::Beta{beta},  
9  fvgrid::VolumeCentered1D{}  
10 );  
11  
12 const Axis random_axis =  
13 fvgrid::random_axis_1d<Scalar>(nvol, xinit, xfinal, seed);
```

A comparação é instrutiva porque apenas a distribuição dos volumes muda. A malha uniforme deve ter comprimentos iguais; a malha de Roberts deve variar suavemente; a malha aleatória pode apresentar irregularidade maior.

Métricas calculadas

As medidas são obtidas a partir de `axis.cell_length(p)`. O programa calcula a soma dos comprimentos, o erro de fechamento, os menores e maiores volumes, a razão global entre maior e menor comprimento, a maior razão local entre vizinhos e um desvio relativo dos comprimentos.

O núcleo da inspeção é uma varredura sobre os volumes:

varredura dos comprimentos

```
1 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
2   const Scalar dx = axis.cell_length(p);
3
4   dx_values[p] = dx;
5   metrics.sum_dx += dx;
6
7   if (dx < metrics.min_dx) {
8     metrics.min_dx = dx;
9     metrics.min_index = p;
10  }
11
12  if (dx > metrics.max_dx) {
13    metrics.max_dx = dx;
14    metrics.max_index = p;
15  }
16 }
```

A razão global $\Delta x_{\max}/\Delta x_{\min}$ indica quanto a malha se afastou do caso uniforme. Já a razão local compara volumes vizinhos:

razão local

```
1 const Scalar local_ratio = (left > right)
2   ? left / right
3   : right / left;
```

Essa segunda medida é importante porque uma malha pode ter uma razão global moderada e, ainda assim, apresentar um salto local indesejado.

O que observar

Na malha uniforme, as razões devem ficar próximas de (1). Na malha de Roberts, espera-se uma variação regular. Na malha aleatória, os indicadores podem crescer mais, especialmente a razão local entre volumes vizinhos.

observação

Métrica não é veredito

As métricas deste exemplo são alertas geométricos. Elas ajudam a encontrar irregularidades, mas não substituem testes numéricos com a equação que será resolvida.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_quality1D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 1.8

Exportação CSV

O exemplo `man_axis1DCSV.cc` encerra a parte 1D mostrando como exportar os dados geométricos de uma malha para um arquivo CSV. O formato é simples, pode ser aberto em planilhas e também pode ser lido por scripts externos para inspeção rápida. A malha usada no exemplo é uma malha de Roberts. A novidade da seção, porém, não está na geração da malha, mas na gravação dos seus dados em arquivo.

Novidade do exemplo

O nome do arquivo e seu caminho absoluto são definidos antes da gravação:

arquivo CSV

```
1 const std::filesystem::path csv_filename{"axis1d_roberts.csv"};
2 const std::filesystem::path csv_path =
3   std::filesystem::absolute(csv_filename);
```

A chamada `std::filesystem::absolute` é usada para imprimir o caminho completo do arquivo. Isso evita ambiguidade sobre onde o CSV foi criado, especialmente quando o exemplo é executado por um alvo do CMake.

Trecho essencial

O arquivo grava uma linha de cabeçalho e, em seguida, uma linha para cada volume:

gravação do CSV

```
1 file << "volume,"
2 << "face_oeste,"
3 << "centro,"
4 << "face_leste,"
5 << "dx,"
6 << "distancia_centro_face_oeste,"
7 << "distancia_face_leste_centro\n";
8
9 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
10 const Scalar west = axis.west_face(p);
11 const Scalar center = axis.center(p);
12 const Scalar east = axis.east_face(p);
13
14 file << p << ','
15     << west << ','
16     << center << ','
17     << east << ','
18     << axis.cell_length(p) << ','
19     << center - west << ','
20     << east - center
21     << '\n';
22 }
```

Cada linha representa um volume. As colunas registram a face oeste, o centro, a face leste, o comprimento Δx_p e as distâncias entre centro e faces. Esses dados são suficientes para verificar a geometria básica da malha fora do programa.

O que observar

O programa imprime no terminal um resumo da malha e o caminho completo do arquivo gerado. O CSV normalmente será criado no diretório de execução do alvo, não necessariamente na pasta manual/. Por isso, a linha com o caminho absoluto é parte importante da saída.

observação

CSV como inspeção

O CSV é adequado para conferência rápida e pós-processamento simples. Para grandes malhas ou visualização científica, formatos específicos, como VTK, são mais apropriados.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_axis1DCSV
```

Após a execução, procure na saída do terminal a linha com o caminho completo do arquivo:

caminho impresso

```
Caminho completo: /caminho/do/repositorio/build/axis1d_roberts.csv
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



02

Malha 2D

eixos, células, áreas, índices e campos escalares

Os exemplos deste capítulo apresentam os recursos fundamentais da FVGridMaker para malhas bidimensionais estruturadas. A ideia central é reaproveitar o que foi visto nas malhas 1D: uma malha 2D é construída pela combinação de dois eixos, um em cada direção lógica. A partir deles, a biblioteca fornece centros, comprimentos locais, áreas, medidas físicas, índices lineares e informações geométricas das células.

A sequência começa por uma malha cartesiana uniforme, usada para comparar área lógica, área cartesiana e medida física. Em seguida, aparece a indexação linear das células, necessária para armazenar campos em vetores. Depois, a malha passa a ser não uniforme nas duas direções. Com a geometria e a indexação já estabelecidas, o capítulo mostra o armazenamento de um campo escalar por célula. Por fim, o exemplo em coordenadas polares separa a malha lógica da geometria física e mostra por que a medida física nem sempre coincide com a área lógica.

Recomenda-se executar os programas na ordem em que aparecem. A parte 2D acrescenta uma direção, mas também introduz uma ideia nova: uma célula passa a ser identificada tanto por índices lógicos (i, j) quanto por um índice linear k , usado quando os dados são armazenados em vetores.

seção 2.1

Áreas em malha cartesiana

O primeiro exemplo bidimensional constrói uma malha cartesiana estruturada a partir de dois eixos 1D. O eixo x define as divisões horizontais, o eixo y define as divisões verticais, e cada célula da malha passa a ser identificada por um par de índices (i, j) .

O exemplo `man_cartesianArea2D.cc` usa uma malha simples para comparar três grandezas geométricas fornecidas pela FVGridMaker: área lógica, área cartesiana e medida física. Em coordenadas cartesianas, essas três quantidades coincidem. Essa igualdade é importante como ponto de partida, porque deixará mais clara a diferença que aparecerá no exemplo em coordenadas polares.

Estrutura básica da malha 2D

A malha 2D é construída a partir de dois objetos do tipo `Axis1D`. O programa usa os mesmos tipos básicos da parte 1D e acrescenta o tipo da malha estruturada bidimensional:

tipos usados

```
1 using Scalar = fvgrid::Real;
2 using Axis = fvgrid::BasicAxis1D<Scalar>;
3 using Grid = fvgrid::BasicStructuredGrid2D<Scalar>;
```

O tipo `Axis` representa cada eixo unidimensional. O tipo `Grid` representa a malha 2D construída pela combinação desses dois eixos.

Neste exemplo, o domínio retangular é dado por $0 \leq x \leq 2$ e $-0,5 \leq y \leq 1$. O eixo x é dividido em 4 volumes, e o eixo y em 3 volumes:

domínio 2D

```
1 const fvgrid::Size nvol_x = 4;
2 const fvgrid::Size nvol_y = 3;
3
4 const Scalar xinit = Scalar{0.0};
5 const Scalar xfinal = Scalar{2.0};
6
7 const Scalar yinit = Scalar{-0.5};
8 const Scalar yfinal = Scalar{1.0};
```

Como a malha é cartesiana e uniforme nas duas direções, os comprimentos locais esperados são

$$\Delta x = \frac{2 - 0}{4} = 0,5, \quad \text{e} \quad \Delta y = \frac{1 - (-0,5)}{3} = 0,5.$$

Logo, cada célula deve ter área 0,25, e a soma das áreas deve recuperar a área total do retângulo:

$$A = (2 - 0) [1 - (-0,5)] = 3.$$

Construção da malha

A construção começa com dois eixos 1D uniformes:

eixos 1D

```
1  const Axis x_axis = fvgrid::uniform_axis_1d<Scalar>(
2  nvol_x,
3  xinit,
4  xfinal
5  );
6
7  const Axis y_axis = fvgrid::uniform_axis_1d<Scalar>(
8  nvol_y,
9  yinit,
10 yfinal
11 );
```

Em seguida, esses dois eixos são combinados para formar a malha 2D:

malha 2D

```
1  const Grid grid{x_axis, y_axis};
```

Essa linha é a primeira ideia importante da parte 2D: uma malha estruturada bidimensional pode ser vista como o produto de dois eixos unidimensionais. Depois que o objeto `grid` é criado, a biblioteca passa a fornecer consultas específicas para a geometria em duas direções.

Resumo geométrico

O programa imprime algumas propriedades globais da malha:

resumo da malha

```
1  std::cout << "sistema de coordenadas : "
2  << grid.coordinate_system_name() << '\n';
3  std::cout << "coordenadas lógicas : "
4  << grid.first_coordinate_name() << ", "
5  << grid.second_coordinate_name() << '\n';
6  std::cout << "número de células em x : " << grid.num_cells_x() << '\n';
7  std::cout << "número de células em y : " << grid.num_cells_y() << '\n';
8  std::cout << "número total de células: " << grid.num_cells() << '\n';
9  std::cout << "xmin : " << grid.xmin() << '\n';
10 std::cout << "xmax : " << grid.xmax() << '\n';
11 std::cout << "ymin : " << grid.ymin() << '\n';
12 std::cout << "ymax : " << grid.ymax() << '\n';
```

As funções `grid.num_cells_x()` e `grid.num_cells_y()` retornam o número de células em cada direção. A função `grid.num_cells()` retorna o número total de células. Neste caso, $N = N_x N_y = 4 \cdot 3 = 12$. Os limites do domínio são obtidos por `grid.xmin()`, `grid.xmax()`, `grid.ymin()` e `grid.ymax()`.

Áreas por célula

Cada célula é percorrida pelo par de índices (i, j) . O índice i identifica a posição na direção x , e o índice j identifica a posição na direção y :

percurso das células

```

1  for (fvgrid::Size j = 0; j < grid.num_cells_y(); ++j) {
2  for (fvgrid::Size i = 0; i < grid.num_cells_x(); ++i) {
3  const Scalar dx = grid.x_cell_length(i);
4  const Scalar dy = grid.y_cell_length(j);
5
6  const Scalar logical_area = grid.cell_logical_area(i, j);
7  const Scalar cartesian_area = grid.cell_area(i, j);
8  const Scalar measure = grid.cell_measure(i, j);
9
10 sum_logical_area += logical_area;
11 sum_cartesian_area += cartesian_area;
12 sum_measure += measure;
13 }
14 }
```

As funções `grid.x_cell_length(i)` e `grid.y_cell_length(j)` retornam os comprimentos locais da célula nas direções x e y . Em uma malha cartesiana, a área da célula é $A_{ij} = \Delta x_i \Delta y_j$. No exemplo, essa mesma área aparece por três chamadas: `grid.cell_logical_area(i, j)`, `grid.cell_area(i, j)` e `grid.cell_measure(i, j)`. Em coordenadas cartesianas, as três grandezas devem ter o mesmo valor. Por isso, este exemplo funciona como referência para os próximos: quando o sistema de coordenadas mudar, essa coincidência poderá deixar de ocorrer.

Consistência geométrica

O programa acumula as três somas e compara cada uma delas com a área analítica do retângulo:

teste de consistência

```

1  const Scalar expected_area = grid.length_x() * grid.length_y();
2
3  const Scalar error_logical_area =
4  std::abs(sum_logical_area - expected_area);
5
6  const Scalar error_cartesian_area =
7  std::abs(sum_cartesian_area - expected_area);
8
9  const Scalar error_measure =
10 std::abs(sum_measure - expected_area);
```

A área esperada é calculada por $A = (x_{\max} - x_{\min})(y_{\max} - y_{\min})$. Neste caso, $A = 3$. Como os cálculos são feitos em ponto flutuante, o programa compara os erros com uma tolerância pequena, em vez de exigir igualdade exata.

Além das somas globais, o exemplo também calcula a maior diferença local entre as três medidas:

diferença local entre medidas

```
1 const Scalar difference_1 = std::abs(logical_area - cartesian_area);
2 const Scalar difference_2 = std::abs(logical_area - measure);
3 const Scalar difference_3 = std::abs(cartesian_area - measure);
4
5 max_difference = std::max(max_difference, difference_1);
6 max_difference = std::max(max_difference, difference_2);
7 max_difference = std::max(max_difference, difference_3);
```

Esse teste verifica se a igualdade entre área lógica, área cartesiana e medida física vale célula a célula, não apenas na soma total.

observação

Área lógica, área cartesiana e medida física

Em coordenadas cartesianas, a área lógica, a área cartesiana e a medida física coincidem. Essa igualdade não deve ser tomada como regra geral para qualquer sistema de coordenadas. Ela vale aqui porque a malha lógica e a malha física usam as mesmas coordenadas x e y .

O que este exemplo fixa

Este exemplo estabelece a base da parte 2D do manual. A partir de agora, uma célula será identificada por (i, j) , terá comprimentos locais Δx_i e Δy_j , e poderá ter grandezas geométricas associadas, como área lógica, área cartesiana e medida física. Nos próximos exemplos, esses conceitos não serão repetidos em detalhe. O texto passará a destacar apenas as novidades: indexação linear, malhas não uniformes, armazenamento de campos escalares e coordenadas polares. Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_cartesianArea2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.2

Numeração linear 2D

O exemplo `man_linearIndex2D.cc` mostra como a `FVGridMaker` converte o par de índices lógicos (i, j) de uma célula 2D em um índice linear k . Essa conversão é necessária quando dados por célula são armazenados em vetores unidimensionais, como ocorre com campos escalares, coeficientes de equações discretizadas ou resíduos.

A geometria usada é a mesma do exemplo anterior: uma malha cartesiana estruturada formada por dois eixos 1D. A novidade desta seção não está nas áreas ou medidas físicas, mas na numeração das células.

Novidade do exemplo

A `FVGridMaker` usa ordenação *row-major*. Nesse padrão, o índice i , associado à direção x , varia mais rapidamente, enquanto j , associado à direção y , identifica a linha da malha. A fórmula é $k = jN_x + i$, em que N_x é o número de células na direção x .

No código, o índice linear é obtido pela função `grid.linear_cell_index(i, j)`:

índice linear

```
1 const fvgrid::Size k_library = grid.linear_cell_index(i, j);
2 const fvgrid::Size k_formula = j * grid.num_cells_x() + i;
3 const bool matches = (k_library == k_formula);
```

A variável `k_library` contém o índice retornado pela biblioteca. A variável `k_formula` contém o índice calculado diretamente pela fórmula row-major. O teste `matches` verifica se os dois valores coincidem.

Trecho essencial

O programa percorre todas as células da malha usando dois laços:

percurso das células

```
1 for (fvgrid::Size j = 0; j < grid.num_cells_y(); ++j) {
2   for (fvgrid::Size i = 0; i < grid.num_cells_x(); ++i) {
3     const fvgrid::Size k_library = grid.linear_cell_index(i, j);
4     const fvgrid::Size k_formula = j * grid.num_cells_x() + i;
5
6     const bool matches = (k_library == k_formula);
7
8     all_indices_match = all_indices_match && matches;
9   }
10 }
```

A ordem dos laços reforça a leitura da malha por linhas: para cada valor de j , percorrem-se todos os valores de i . Assim, em uma malha com $N_x = 4$, os índices da primeira linha são (0,1,2,3), os da segunda são (4,5,6,7), e assim por diante.

O que observar

Na saída, cada linha da tabela associa uma célula (i, j) ao índice linear k . O valor retornado por `grid.linear_cell_index(i, j)` deve coincidir com $jN_x + i$ para todas as células.

observação

Por que usar índice linear?

Embora a malha seja naturalmente descrita por dois índices, muitos dados numéricos são armazenados em vetores. O índice linear k faz a ponte entre a geometria bidimensional da célula e a estrutura linear usada no armazenamento computacional.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_linearIndex2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.3

Malha 2D com faces explícitas

O exemplo `man_nonUniformGrid2D.cc` mostra como construir uma malha cartesiana 2D não uniforme a partir de faces fornecidas explicitamente nas duas direções. A ideia é a mesma usada nas malhas 1D customizadas: o usuário informa as coordenadas das faces, e a FVGridMaker monta os eixos e a geometria da malha.

A diferença em relação aos exemplos anteriores está nos espaçamentos. Antes, a malha 2D era uniforme nas direções x e y . Agora, os comprimentos Δx_i e Δy_j variam com os índices. A interface de consulta, porém, permanece a mesma.

Novidade do exemplo

As faces são fornecidas por dois vetores, um para cada direção:

faces em x e y

```
1  const std::vector<Scalar> x_faces{
2    Scalar{0.00},
3    Scalar{0.15},
4    Scalar{0.50},
5    Scalar{1.10},
6    Scalar{2.00}
7  };
8
9  const std::vector<Scalar> y_faces{
10   Scalar{-0.50},
11   Scalar{-0.10},
12   Scalar{0.35},
13   Scalar{1.00}
14  };
```

Como há 5 faces em x , a malha tem 4 células nessa direção. Como há 4 faces em y , ela tem 3 células nessa direção. O número total de células é, portanto, $N = N_x N_y = 4 \cdot 3 = 12$.

Trecho essencial

Cada vetor de faces é usado para construir um eixo 1D. Em seguida, os dois eixos são combinados para formar a malha 2D:

construção da malha 2D não uniforme

```
1  const Axis x_axis{x_faces};
2  const Axis y_axis{y_faces};
3  const Grid grid{x_axis, y_axis};
```

Essa é a ideia central aqui: uma malha 2D estruturada continua sendo a combinação de dois eixos 1D. A não uniformidade aparece primeiro nos eixos; e, propaga-se para as células 2D.

O que observar

Os comprimentos locais são obtidos pelas mesmas chamadas já usadas no exemplo cartesiano:

comprimentos locais e medida

```
1 const Scalar dx = grid.x_cell_length(i);
2 const Scalar dy = grid.y_cell_length(j);
3 const Scalar measure = grid.cell_measure(i, j);
```

Em coordenadas cartesianas, a medida física da célula continua sendo $A_{ij} = \Delta x_i \Delta y_j$. A diferença é que agora Δx_i e Δy_j não são constantes. Por isso, as áreas das células variam ao longo da malha. O programa também verifica se a soma das medidas físicas das células recupera a área total do domínio retangular:

$$\sum_{j=0}^{N_y-1} \sum_{i=0}^{N_x-1} A_{ij} = (x_{\max} - x_{\min})(y_{\max} - y_{\min}).$$

observação

Não uniformidade em duas direções

Este exemplo mostra que a não uniformidade 2D pode ser entendida como a composição de duas não uniformidades 1D. A célula (i, j) herda seu comprimento em x do eixo x , seu comprimento em y do eixo y , e sua medida física resulta do produto desses comprimentos no caso cartesiano.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_nonUniformGrid2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.4

Campo escalar 2D

O exemplo `man_scalarField2D.cc` mostra como armazenar um campo escalar definido nas células de uma malha 2D. A geometria e a indexação linear já foram apresentadas nas seções anteriores. Agora, elas são usadas para preencher um vetor com uma entrada por célula.

A ideia é simples: cada célula é identificada logicamente por (i, j) , mas o valor do campo é armazenado na posição linear k . A ligação entre essas duas representações é feita por `grid.linear_cell_index(i, j)`.

Novidade do exemplo

O campo usado no programa é $\phi(x, y) = 1 + x + 2y$. Essa função é avaliada no centro de cada célula. O programa define a função em uma rotina auxiliar:

campo escalar

```
1 Scalar field_value(Scalar x, Scalar y) {  
2   return Scalar{1.0} + x + Scalar{2.0} * y;  
3 }
```

A malha usada é cartesiana e não uniforme, como no exemplo anterior. Isso torna a tabela mais informativa, pois as células não têm todas a mesma medida física.

Trecho essencial

O vetor do campo é criado com uma entrada para cada célula:

vetor do campo

```
1 std::vector<Scalar> phi(grid.num_cells(), Scalar{0.0});
```

Em seguida, o programa percorre todas as células, calcula o índice linear k , obtém o centro da célula e armazena o valor de ϕ :

preenchimento do campo

```
1 for (fvgrid::Size j = 0; j < grid.num_cells_y(); ++j) {  
2   for (fvgrid::Size i = 0; i < grid.num_cells_x(); ++i) {  
3     const fvgrid::Size k = grid.linear_cell_index(i, j);  
4  
5     const Scalar x = grid.x_center(i);  
6     const Scalar y = grid.y_center(j);  
7  
8     phi[k] = field_value(x, y);  
9   }  
10 }
```

Esse trecho fixa o padrão de armazenamento usado em muitos códigos numéricos: a geometria continua sendo acessada por (i, j) , mas os dados são guardados em um vetor linear.

Integral discreta

Depois de preencher o campo, o programa calcula uma integral discreta por soma ponderada:

$$\int_{\Omega} \phi, d\Omega \approx \sum_{j=0}^{N_y-1} \sum_{i=0}^{N_x-1} \phi_{ij} A_{ij}.$$

No código, A_{ij} é obtido por `grid.cell_measure(i, j)`:

contribuição por célula

```
1 const fvgrid::Size k = grid.linear_cell_index(i, j);
2
3 const Scalar measure = grid.cell_measure(i, j);
4 const Scalar contribution = phi[k] * measure;
5
6 sum_measure += measure;
7 discrete_integral += contribution;
```

A quantidade `contribution` representa a contribuição da célula para a integral discreta. O programa também calcula a média discreta dividindo a integral pela soma das medidas.

O que observar

Como o campo escolhido é linear, a regra do ponto médio por célula integra exatamente em cada retângulo da malha, salvo erro de arredondamento. Por isso, a integral discreta deve coincidir com o valor analítico calculado no programa.

observação

Geometria e dados

A malha fornece a geometria: centros, medidas e índices. O vetor `phi` armazena os dados associados às células. A função `grid.linear_cell_index(i, j)` faz a ligação entre essas duas partes.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_scalarField2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.5

Malha polar 2D

O exemplo `man_polarGrid2D.cc` mostra uma malha estruturada em coordenadas polares. Até aqui, as coordenadas lógicas da malha coincidiam com as coordenadas físicas cartesianas. Agora, a malha é construída no plano lógico (r, θ) , mas representa um setor anular no plano físico xy .

Essa é a principal novidade da seção: a célula continua sendo identificada por (i, j) , mas sua medida física não é mais dada simplesmente pela área lógica $\Delta r_i, \Delta \theta_j$.

Novidade do exemplo

O domínio lógico é $1 \leq r \leq 2$ com $0 \leq \theta \leq \frac{\pi}{2}$. O programa define dois eixos uniformes, um para r e outro para θ :

eixos polares

```
1  const Axis r_axis = fvgrid::uniform_axis_1d<Scalar>(  
2  nvol_r,  
3  r_inner,  
4  r_outer  
5  );  
6  
7  const Axis theta_axis = fvgrid::uniform_axis_1d<Scalar>(  
8  nvol_theta,  
9  theta_begin,  
10 theta_end  
11 );
```

Esses eixos formam uma malha retangular no espaço lógico (r, θ) . No plano físico, porém, essa malha corresponde a um setor anular.

Trecho essencial

A diferença em relação à malha cartesiana aparece na construção do objeto `Grid`. Agora, além dos dois eixos, o programa informa explicitamente o sistema de coordenadas:

malha em coordenadas polares

```
1  const Grid grid(  
2  r_axis,  
3  theta_axis,  
4  fvgrid::BasicPolarCoordinates2D<Scalar>{}  
5  );
```

O terceiro argumento indica que os pontos físicos são obtidos pelo mapeamento polar $x = r \cos \theta$ e $y = r \sin \theta$. Assim, as coordenadas armazenadas nos eixos continuam sendo r e θ , mas a interpretação geométrica da célula passa a ser feita no plano físico.

Área lógica e medida física

Para cada célula, o programa compara duas grandezas:

$$A_{ij}^{\log} = \Delta r_i \Delta \theta_j$$

e

$$A_{ij}^{\text{fis}} = \frac{1}{2} (r_{i+1}^2 - r_i^2) (\theta_{j+1} - \theta_j).$$

A primeira é a área do retângulo lógico em (r, θ) . A segunda é a área física do setor anular correspondente no plano xy . Em coordenadas polares, essas duas quantidades não coincidem. No código, a comparação aparece por meio das chamadas:

área lógica e medida física

```
1 const Scalar logical_area = grid.cell_logical_area(i, j);
2 const Scalar measure = grid.cell_measure(i, j);
```

A função `grid.cell_logical_area(i, j)` mede a célula no espaço lógico, enquanto a função `grid.cell_measure(i, j)` retorna a medida física associada ao sistema de coordenadas escolhido.

Centro lógico e ponto físico

O programa também mostra o centro lógico da célula e o ponto físico correspondente:

centro lógico e ponto físico

```
1 const Scalar r = grid.first_center(i);
2 const Scalar theta = grid.second_center(j);
3
4 const auto physical_center = grid.physical_cell_center(i, j);
```

As chamadas `grid.first_center(i)` e `grid.second_center(j)` retornam as coordenadas lógicas do centro. Já `grid.physical_cell_center(i, j)` aplica o mapeamento polar e devolve o ponto correspondente no plano físico.

O que observar

A soma das áreas lógicas recupera a área do retângulo lógico, $(r_{\max} - r_{\min})(\theta_{\max} - \theta_{\min})$, enquanto a soma das medidas físicas recupera a área do setor anular, $\frac{1}{2} (r_{\max}^2 - r_{\min}^2) (\theta_{\max} - \theta_{\min})$.

Esses dois valores são diferentes. Essa diferença é esperada e confirma que a FVGridMaker está distinguindo corretamente a malha lógica da geometria física.

observação

Malha lógica e geometria física

Em coordenadas cartesianas, área lógica e medida física coincidiam. Em coordenadas polares, não. A malha continua estruturada em (r, θ) , mas a medida física das células deve respeitar o mapeamento para o plano xy .

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_polarGrid2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.6

Inspeção da malha 2D

O exemplo `man_structuredGrid2D.cc` retoma a malha cartesiana da seção de abertura, mas com outro objetivo: percorrer e imprimir, de uma só vez, todas as informações geométricas que a FVGridMaker oferece para uma malha 2D. A geometria é a mesma de antes ($0 \leq x \leq 2$ e $-0,5 \leq y \leq 1$, com 4 volumes em x e 3 em y), e a construção também: dois eixos uniformes combinados em um Grid cartesiano.

A novidade desta seção não está na construção da malha, e sim na inspeção. Até aqui, o capítulo destacou áreas, índices e campos isoladamente. Este exemplo serve como referência de consulta: reúne nomes de coordenadas, padrões dos eixos, contagem de células e de faces, limites, comprimentos e a geometria volume a volume.

Novidade do exemplo

A primeira novidade é que o objeto `Grid` dá acesso aos eixos que o formaram. Cada eixo continua sendo um `BasicAxis1D` e responde às mesmas consultas da parte 1D, inclusive ao nome do padrão de distribuição:

padrões dos eixos

```
1 std::cout << "padrão do eixo x : "  
2           << grid.first_axis().pattern_name() << '\n';  
3 std::cout << "padrão do eixo y : "  
4           << grid.second_axis().pattern_name() << '\n';
```

As chamadas `grid.first_axis()` e `grid.second_axis()` devolvem os eixos x e y . Com elas, qualquer recurso já visto em 1D fica disponível diretamente a partir da malha 2D.

Faces por direção

As coordenadas das faces são consultadas por direção. Como há 4 volumes em x , existem 5 faces em x ; como há 3 volumes em y , existem 4 faces em y :

faces em x

```
1 for (fvgrid::Size i = 0; i < grid.num_faces_x(); ++i) {  
2     std::cout << std::right  
3             << std::setw(id_width) << i  
4             << std::setw(value_width) << grid.x_face(i)  
5             << '\n';  
6 }
```

A chamada `grid.num_faces_x()` retorna o número de faces na direção x , e `grid.x_face(i)` retorna a coordenada da face de índice i . As funções `grid.num_faces_y()` e `grid.y_face(j)` cumprem o mesmo papel na direção y .

Geometria volume a volume

Por fim, o programa percorre as células e imprime, para cada volume, os índices lógicos (i, j) , o índice linear k , as quatro faces que o delimitam, o centro, a área lógica e a medida física:

geometria dos volumes

```
1 const fvgrid::Size k = grid.linear_cell_index(i, j);
2
3 std::cout << grid.x_face(i) << grid.x_center(i) << grid.x_face(i + 1)
4           << grid.y_face(j) << grid.y_center(j) << grid.y_face(j + 1)
5           << grid.cell_logical_area(i, j)
6           << grid.cell_measure(i, j);
```

As faces oeste e leste de um volume são `grid.x_face(i)` e `grid.x_face(i + 1)`; as faces sul e norte, `grid.y_face(j)` e `grid.y_face(j + 1)`.

O que observar

Como o sistema é cartesiano, a área lógica e a medida física coincidem volume a volume, e a soma das medidas reproduz a área do retângulo. Essa é a mesma verificação da seção de áreas, repetida aqui para fechar a inspeção. A diferença entre área lógica e medida física só aparecerá quando o sistema de coordenadas deixar de ser cartesiano, como nos exemplos seguintes.

observação

Uma malha, muitas consultas

A mesma malha responde a consultas globais (nomes, contagens, limites, comprimentos) e a consultas locais (faces, centros, áreas e medidas por célula). Acessar os eixos com `first_axis()` e `second_axis()` reaproveita, em 2D, toda a interface 1D.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_structuredGrid2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.7

Qualidade e exportação VTK

O exemplo `man_qualityAndVTK2D.cc` encerra a parte 2D com duas tarefas que aparecem com frequência depois de construir uma malha: medir sua qualidade e gravá-la em um arquivo para inspeção visual. É o equivalente bidimensional da exportação CSV vista em 1D, agora no formato VTK, próprio para visualizadores científicos.

A malha usada é uma arruela com um setor angular removido, em coordenadas polares. Essa escolha não é nova; ela aparece com mais detalhe na seção anterior. Aqui, ela serve apenas de pretexto: por não ser um retângulo no plano físico, produz um arquivo VTK mais interessante de abrir no ParaView. As duas novidades da seção são o relatório de qualidade e a chamada de exportação.

Novidade do exemplo

A primeira novidade é o relatório de qualidade. Em vez de o programa recalculer medidas à mão, ele pede um relatório pronto ao módulo de qualidade da biblioteca:

relatório de qualidade

```
1 const auto report = fvgrid::Quality2D::evaluate(grid);
```

A chamada `fvgrid::Quality2D::evaluate(grid)` percorre a malha e devolve uma estrutura com indicadores já calculados. Entre eles estão a menor e a maior medida física das células, com a razão entre elas; a menor e a maior área lógica (computacional), também com sua razão; e as maiores razões entre células adjacentes, separadas por direção. Esses campos são lidos diretamente, sem laços:

leitura dos indicadores

```
1 std::cout << "razão max/min (medida) : "  
2 << report.cell_measure_ratio << '\n';  
3 std::cout << "razão radial adjacente : "  
4 << report.max_adjacent_cell_measure_ratio_first_direction << '\n';  
5 std::cout << "razão angular adjacente: "  
6 << report.max_adjacent_cell_measure_ratio_second_direction << '\n';
```

Como na parte 1D, esses indicadores não decidem se a malha é adequada para um problema; eles apenas quantificam regularidade e concentração.

Trecho essencial

A segunda novidade é a exportação. Gravar a malha em VTK legacy é uma única linha:

exportação VTK

```
1 const std::filesystem::path vtk_file{"malha_qualidade_2d.vtk"};  
2 fvgrid::write_vtk(grid, vtk_file);
```

A função `fvgrid::write_vtk` recebe a malha e o caminho do arquivo e escolhe, internamente, a representação VTK adequada ao sistema de coordenadas. A consulta a função `grid.vtk_rectilinear()` indica qual representação será usada: *rectilinear* para malhas alinhadas aos eixos, ou *structured* quando há um mapeamento físico, como nesta malha polar.

O que observar

O programa imprime o relatório, grava o arquivo e, ao final, confirma que o VTK foi criado e não está vazio. As razões devem ser maiores ou iguais a 1, e as medidas, positivas. O arquivo gerado pode ser aberto no ParaView, onde a arruela aparece com o setor removido.

observação**CSV e VTK**

Em 1D, o CSV foi suficiente para conferência rápida. Em 2D, com geometria física, o VTK preserva a forma das células e permite visualização. A biblioteca escolhe entre as representações *rectilinear* e *structured* conforme o sistema de coordenadas; ao usuário, basta `write_vtk`.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_qualityAndVTK2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.8

Malha axissimétrica 2D

O exemplo `man_axisymmetricGrid2D.cc` apresenta uma malha em coordenadas axissimétricas cilíndricas. A ideia é a mesma da malha polar: a malha lógica é retangular, mas o sistema de coordenadas faz a medida física da célula deixar de coincidir com a área lógica. A diferença está no significado dessa medida.

Aqui, as coordenadas lógicas são (r, z) : r é a distância ao eixo de simetria e z é a posição ao longo dele. Cada célula retangular no plano (r, z) representa um sólido de revolução em torno do eixo z . Por isso, a medida física da célula não é uma área, e sim um volume.

Novidade do exemplo

A novidade é o terceiro argumento do construtor, que seleciona o sistema axissimétrico:

malha axissimétrica

```
1 const Grid grid{
2   r_axis,
3   z_axis,
4   fvgrid::BasicAxisymmetricCoordinates2D<Scalar>{}
5 };
```

Os eixos `r_axis` e `z_axis` são construídos como qualquer eixo uniforme. O que muda é a interpretação: com `BasicAxisymmetricCoordinates2D`, a célula entre os raios r_i e r_{i+1} e as alturas z_j e z_{j+1} passa a representar a casca cilíndrica obtida pela rotação completa em torno do eixo z . Seu volume é

$$V_{ij} = \pi (r_{i+1}^2 - r_i^2) (z_{j+1} - z_j).$$

Área lógica e medida física

A comparação entre a área lógica e a medida física usa as mesmas chamadas da malha polar:

área lógica e medida física

```
1 const Scalar logical_area = grid.cell_logical_area(i, j);
2 const Scalar measure = grid.cell_measure(i, j);
```

A função `grid.cell_logical_area(i, j)` mede o retângulo $\Delta r_i \Delta z_j$ no plano lógico, enquanto `grid.cell_measure(i, j)` devolve o volume de revolução V_{ij} . As coordenadas do centro lógico vêm de `grid.first_center(i)` e `grid.second_center(j)`, e o ponto mapeado, de `grid.physical_cell_center(i, j)`, que neste sistema permanece no plano meridional (r, z) .

O que observar

A soma das áreas lógicas recupera a área do retângulo lógico, $(r_{\max} - r_{\min})(z_{\max} - z_{\min})$, enquanto a soma das medidas físicas recupera o volume da casca cilíndrica completa,

$$V = \pi (r_{\max}^2 - r_{\min}^2) (z_{\max} - z_{\min}).$$

Esses dois valores são diferentes, e essa diferença é o ponto da seção: em coordenadas axissimétricas, a geometria física carrega o fator de revolução que a área lógica não enxerga.

observação

Área no plano, volume na revolução

Em (r, z) , a área lógica mede o retângulo no plano meridional. A medida física mede o volume gerado ao girar esse retângulo em torno do eixo de simetria. A malha continua estruturada e retangular no plano lógico; é o mapeamento que introduz o volume.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_axisymmetricGrid2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.9

Anular com setor removido

O exemplo `man_incompleteAnnulus2D.cc` mostra uma arruela (um anular) à qual se removeu um setor angular. A novidade não está em um novo gerador de malha, e sim na percepção de que essa geometria já é alcançável com o que o capítulo apresentou: trata-se de uma malha polar usada sobre um intervalo angular parcial.

A malha continua sendo construída em coordenadas polares (r, θ) , como na seção da malha polar. A única mudança conceitual é o intervalo de θ : em vez de varrer a volta inteira, ele cobre apenas a parte que permanece depois de retirar o setor.

Novidade do exemplo

A região mantida é $r \in [1, 2]$ e $\theta \in [\pi/6, 11\pi/6]$, o que corresponde a remover um setor de 60° centrado no eixo x positivo. Os limites angulares são obtidos a partir da abertura removida:

intervalo angular parcial

```
1 const Scalar removed_angle = pi / Scalar{3.0};  
2  
3 const Scalar theta_begin = Scalar{0.5} * removed_angle;  
4 const Scalar theta_end   = Scalar{2.0} * pi - Scalar{0.5} * removed_angle;
```

A construção da malha é idêntica à da malha polar completa: dois eixos uniformes e o sistema de coordenadas polar. O eixo angular apenas começa em `theta_begin` e termina em `theta_end`, em vez de fechar o círculo:

malha polar parcial

```
1 const Grid grid{  
2   r_axis,  
3   theta_axis,  
4   fvgrid::BasicPolarCoordinates2D<Scalar>{}  
5 };;
```

Nada mais precisa mudar. A não uniformidade física, a indexação e as medidas das células seguem funcionando como no anular completo.

Trecho essencial

Como a malha tem muitas células, a tabela imprime apenas uma amostra (cantos e meio), mas a verificação geométrica usa todas elas:

amostra e acumulação

```
1 sum_logical_area += grid.cell_logical_area(i, j);
2 sum_measure      += grid.cell_measure(i, j);
3
4 if (is_sample_cell(i, j, grid.num_cells_x(), grid.num_cells_y())) {
5     const auto mapped_point = grid.physical_cell_center(i, j);
6     // imprime r, theta, x_map, y_map, área lógica e medida
7 }
```

O que observar

A soma das medidas físicas recupera a área do setor anular restante,

$$A^{\text{fis}} = \frac{1}{2} (r_{\text{max}}^2 - r_{\text{min}}^2) (\theta_{\text{end}} - \theta_{\text{begin}}),$$

menor que a área do anular completo exatamente na proporção do setor removido. Ao final, o programa grava a malha em VTK com `fvgrid::write_vtk`, e o arquivo mostra o anular com a abertura.

observação

Geometria nova, código conhecido

Recortar um setor não exigiu uma nova ferramenta: bastou restringir o intervalo de θ na malha polar. Boa parte das geometrias úteis aparece ao reusar um sistema de coordenadas já existente sobre um domínio diferente.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_incompleteAnnulus2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.10

Malha elíptica confocal

O exemplo `man_ellipticGrid2D.cc` dá um passo além dos sistemas de coordenadas prontos. Até aqui, a malha usou sistemas que a biblioteca já conhece: cartesiano, polar e axissimétrico. Agora, a novidade é construir um sistema de coordenadas *próprio* — coordenadas elípticas confocais — e entregá-lo à FVGridMaker.

As coordenadas lógicas são (μ, ν) , e o mapeamento físico é

$$x = a \cosh \mu \cos \nu, \quad y = a \sinh \mu \sin \nu.$$

As linhas de μ constante mapeiam elipses; as de ν constante, hipérboles. A biblioteca não precisa conhecer “coordenadas elípticas” como um tipo nativo: o usuário fornece o mapeamento e as fórmulas de medida.

Novidade do exemplo

O sistema personalizado é montado pela fábrica `make_basic_coordinate_mapping_2d`. Ela recebe o nome do sistema, os nomes das coordenadas lógicas e quatro funções que descrevem a geometria física:

mapeamento personalizado

```
1 const auto elliptic_mapping = fvgrid::make_basic_coordinate_mapping_2d<Scalar>(
2     "EllipticConfocal", "mu", "nu",
3     physical_point,      // (mu, nu) -> ponto físico (x, y)
4     cell_measure,        // célula lógica -> medida física da célula
5     first_face_measure,  // comprimento da face de mu constante
6     second_face_measure, // comprimento da face de nu constante
7     false                // não é rectilinear para o VTK
8 );
9
10 const Grid grid{mu_axis, nu_axis, elliptic_mapping};
```

A primeira função devolve o ponto físico a partir de (μ, ν) , montando um `BasicPhysicalPoint2D`. A segunda recebe a célula lógica (um `BasicCoordinateCell2D`, com os limites em cada direção) e devolve sua medida física. As duas últimas devolvem os comprimentos físicos das faces em cada família. O último argumento informa ao escritor VTK que esta malha não é *rectilinear*.

Trecho essencial

A medida física da célula é dada pela integral analítica do jacobiano elíptico sobre o retângulo (μ, ν) :

medida física da célula

```
1 return (a * a / Scalar{4.0})  
2 * ( delta_nu * (std::sinh(2*mu_max) - std::sinh(2*mu_min))  
3   - delta_mu * (std::sin (2*nu_max) - std::sin (2*nu_min)) );
```

Os comprimentos das faces, por não terem forma fechada simples, são integrados numericamente pela regra de Simpson composta. Esse é o ponto da seção: definido o mapeamento e suas medidas, a célula (i, j) passa a ter geometria elíptica sem que a biblioteca precise de qualquer tipo novo.

O que observar

A soma das medidas físicas das células recupera a integral analítica do jacobiano no domínio inteiro, e o programa ainda confere, célula a célula, a maior diferença entre a medida retornada pela malha e a fórmula analítica. A malha é gravada em VTK com `fvgrid::write_vtk`; no ParaView, as linhas coordenadas aparecem como elipses e hipérboles.

observação**Quando o sistema não é nativo**

Para sistemas que a biblioteca não traz prontos, `make_basic_coordinate_mapping_2d` é o ponto de extensão: o usuário fornece o mapeamento físico e as medidas de célula e de face, e a malha estruturada passa a operar nesse sistema como em qualquer outro.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_ellipticGrid2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



seção 2.11

Interseção e recorte

O exemplo `man_clipGrid2D.cc` muda o foco do capítulo. Todas as seções anteriores trataram de *gerar* uma malha. Esta trata de *operar* sobre malhas já existentes: dadas duas malhas que se sobrepõem em parte, calcular a região comum e recortar uma delas para essa região.

A novidade está no módulo de operações `Operations2D`. Em vez de propriedades de uma única malha, ele expõe operações que combinam malhas e produzem novas malhas ou descrições de domínio.

Novidade do exemplo

O programa cria duas malhas cartesianas parcialmente sobrepostas. A primeira operação extrai a *caixa lógica* de cada uma — a descrição retangular do domínio em coordenadas lógicas:

caixas de domínio

```
1 const auto box_a = fvgrid::Operations2D::domain_box(grid_a);  
2 const auto box_b = fvgrid::Operations2D::domain_box(grid_b);
```

A função `Operations2D::domain_box(grid)` devolve um `BasicLogicalBox2D`. Cada caixa expõe seus intervalos por direção, com `first_interval()` e `second_interval()`, e a área computacional do domínio, com `computational_area()`.

Trecho essencial

A interseção das duas caixas é obtida diretamente. A variante `require_box_intersection` exige que a interseção exista e tenha área positiva, falhando caso contrário; a tolerância trata comparações em ponto flutuante:

interseção e recorte

```
1 const auto intersection_box = fvgrid::Operations2D::require_box_intersection(  
2     grid_a, grid_b, tolerance  
3 );  
4  
5 const Grid clipped_grid = fvgrid::Operations2D::clip_to_logical_box(  
6     grid_a, intersection_box, tolerance  
7 );
```

A chamada `clip_to_logical_box` é o centro do exemplo: ela recorta `grid_a` à caixa comum e devolve uma nova malha estruturada, com seus próprios eixos, células e índices. Neste exemplo, os limites de `grid_b` foram escolhidos para coincidir com faces de `grid_a`, o que torna o recorte exato.

O que observar

A malha recortada é uma malha como qualquer outra: o programa percorre suas células com `linear_cell_index`, `x_center`, `y_center` e `cell_measure`, exatamente como nas seções anteriores. O teste de consistência verifica que a soma das medidas da malha recortada coincide com a área computacional da caixa comum, `intersection_box.computational_area()`. Ao final, a malha recortada é gravada em VTK com `fvgrid::write_vtk`.

observação

Gerar e operar

`Operations2D` acrescenta uma segunda família de tarefas ao capítulo: além de construir malhas, é possível combiná-las. O recorte produz uma malha estruturada nova, que volta a responder a toda a interface 2D já conhecida.

Para executar este exemplo, use:

execução

```
cmake --build build --target run_man_clipGrid2D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



Referências Bibliográficas

- [1] Maliska, C. R. *Transferência de calor e mecânica dos fluidos computacional*. 1ª edição. Rio de Janeiro, RJ, Brasil: LTC Editora, 1994, págs. 1–472 (ver [pág. 10](#))
- [2] Roberts, G. O. “Computational meshes for boundary layer problems”. Em: *Proceedings of the Second International Conference on Numerical Methods in Fluid Dynamics*. Vol. 8. Berlin, Heidelberg: Springer Berlin Heidelberg, 1971, págs. 171–177. DOI: [10.1007/3-540-05407-3_{_}27](https://doi.org/10.1007/3-540-05407-3_{_}27) (ver [pág. 10](#))